

Experiment No.	4
Experiment Title	Binary Classification using Linear and Kernel-Based Models
Student Name	Sathiya Priyan
Register Number	3122247001058
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	August 16, 2026

1 Objective

To classify emails as spam or ham (non-spam) using the Spambase dataset by building and comparing a linear classifier (Logistic Regression) and a margin-based kernel classifier (Support Vector Machine). The experiment covers training baseline models, tuning hyperparameters (regularization type and strength for Logistic Regression; kernel, C and γ for SVM), comparing SVM performance across the Linear, Polynomial, RBF, and Sigmoid kernels, validating the tuned models using 5-fold cross-validation, and analyzing the effect of hyperparameter tuning on classification performance.

2 Problem Statement

Input: 57 numerical features extracted from the content of an email (word-frequency, character-frequency, and capital-letter run-length statistics).

Output: A binary label indicating whether the email is **Spam (1)** or **Ham (0)**.

Task: This is a supervised binary classification problem. Two families of classifiers are trained and compared on the same train/test split:

- **Logistic Regression** — a linear, probabilistic classifier, tuned over the regularization penalty (L1/L2), inverse regularization strength C , and solver.
- **Support Vector Machine (SVM)** — a margin-based classifier, evaluated with Linear, Polynomial, RBF, and Sigmoid kernels and tuned over kernel, C , γ , and (for the polynomial kernel) degree.

3 Dataset Description

Dataset Name	Spambase Dataset
Dataset Source	Kaggle: https://www.kaggle.com/datasets/somesh24/spambase
Number of Samples	4601
Number of Features	57
Number of Classes	2 (0 = Ham, 1 = Spam)
Missing Values	None (<code>df.isnull().sum().sum() = 0</code>)
Train-Test Split	80% / 20% (3680 training samples, 921 test samples), <code>random_state=42</code> , <code>stratify=y</code>

4 Exploratory Data Analysis

Include all relevant visualizations.

- Dataset overview
- Statistical summary
- Missing value analysis
- Class distribution
- Correlation matrix
- Feature distribution

The dataset consists of 4601 emails, each described by 57 numerical features (`df.info()` confirms all 58 columns, including the target, are non-null). `df.describe()` shows the word-frequency and character-frequency features are heavily right-skewed, with most values at or near zero and a small number of large outliers (e.g. `capital_run_length_total` has a mean of 283 but a maximum of 15,841).

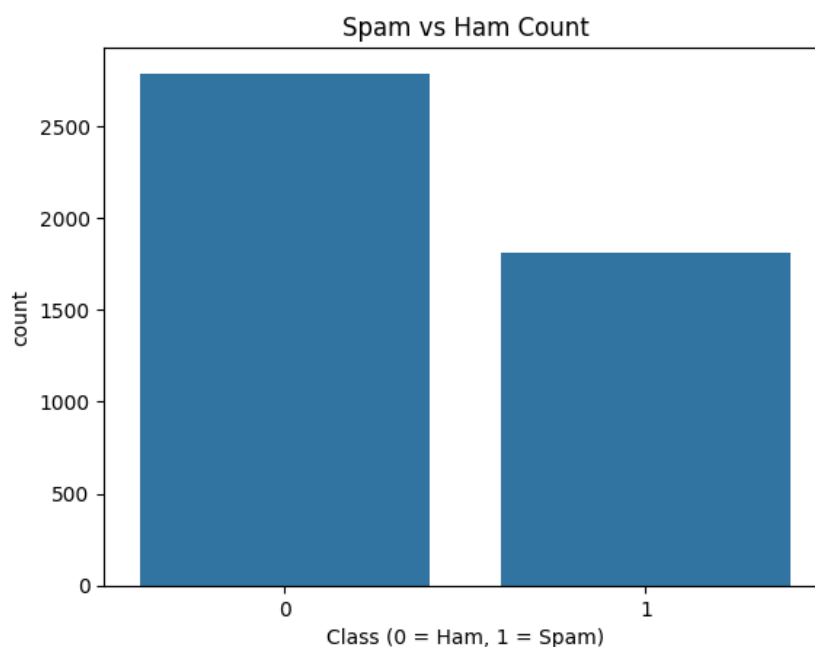


Figure 1: Class distribution of Spam vs Ham emails.

Inference:

The dataset contains 2788 ham emails (class 0) and 1813 spam emails (class 1), i.e. roughly a 60.6%/39.4% split. The classes are moderately imbalanced but not severely so, which is why the train-test split uses stratified sampling (`stratify=y`) to preserve this ratio in both the training and test sets. This level of imbalance is generally manageable for Logistic Regression and SVM without requiring resampling, though it does mean accuracy alone should be interpreted alongside precision/recall.

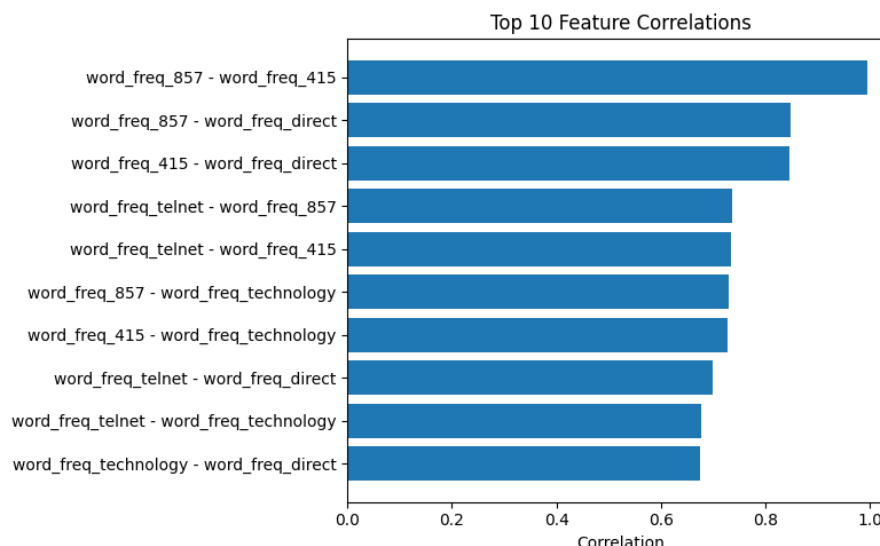


Figure 2: Correlation heatmap of all 58 Spambase columns (57 features + target).

Inference:

The heatmap shows that most word-frequency features are weakly correlated with one another, consistent with them representing frequencies of different, largely independent words. A visible band of stronger correlation appears among related word-frequency groups (e.g. business/technical terms such as `word_freq_650`, `word_freq_technology`, `word_freq_labs`), which tend to co-occur in similar (often legitimate, work-related) emails. The capital-run-length features and several `char_freq` features (such as `%21` for ‘!’ and `%24` for ‘\$’) show a visible relationship with the target column, consistent with these being common markers of promotional/spam content.

5 Data Preprocessing

Explain:

- Missing value handling
- Encoding
- Feature scaling
- Train-test split
- Feature engineering (if applicable)

Missing value handling: The notebook checked for missing values using `df.isnull().sum().sum()` which returned 0 — no missing-value imputation was required.

Encoding: The target column `class` was already numerically encoded (0 = Ham, 1 = Spam) in the source CSV, so no additional label encoding was performed.

Feature and target separation: Features (`X`, shape 4601×57) were separated from the target (`y`, shape 4601) by dropping the `class` column.

Train-test split: An 80/20 split was performed using `train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)`, giving 3680 training samples and 921 test samples, with class proportions preserved via stratification.

Feature scaling: A `StandardScaler` was fit on the training features only (`scaler.fit_transform(X_train)`) and then applied to transform the test features (`scaler.transform(X_test)`), so that every feature has zero mean and unit variance on the training set. This is essential for both Logistic Regression (regularization penalizes coefficient magnitude, which is scale-dependent) and SVM (the margin and RBF/polynomial kernel computations depend directly on feature distances).

Feature engineering: No additional feature engineering (e.g. polynomial feature expansion, feature selection) was performed in the notebook; all 57 original features were used.

6 Machine Learning Algorithm

6.1 Logistic Regression

Logistic Regression is a probabilistic linear classifier for binary classification. It models the probability that a sample belongs to the positive class using the sigmoid function:

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-(w^T x + b)}}$$

A threshold (typically 0.5) converts the predicted probability into a class label. The model is trained by minimizing the log-loss (cross-entropy loss) between predicted probabilities and true labels.

Regularization: L1 (Lasso) regularization shrinks some coefficients exactly to zero, performing implicit feature selection; L2 (Ridge) regularization shrinks all coefficients but keeps every feature, generally improving generalization. The inverse regularization strength C controls the trade-off: small C means strong regularization (simpler model), large C means weak regularization (more complex, higher-variance model). The `solver` determines the optimization algorithm used — `liblinear` is suited to smaller datasets and supports both L1 and L2 penalties, while `saga` scales better to larger datasets and also supports both penalties.

Advantages: Fast to train, highly interpretable coefficients, low variance, works well when classes are close to linearly separable.

Limitations: Cannot capture non-linear decision boundaries directly; performance degrades if the true relationship between features and the target is strongly non-linear.

6.2 Support Vector Machine (SVM)

SVM is a margin-based classifier that finds the hyperplane that maximizes the margin between the two classes. Only a subset of training points close to the decision boundary — the **support vectors** — determine the position of the hyperplane.

Kernels used in this experiment:

- **Linear** — suitable for linearly separable data, equivalent to no kernel transformation.
- **Polynomial** — captures polynomial feature interactions, controlled by the `degree` parameter.
- **RBF (Radial Basis Function)** — handles complex, non-linear decision boundaries by implicitly mapping data to an infinite-dimensional space.
- **Sigmoid** — behaves similarly to a neural network activation function.

Hyperparameters: C controls the trade-off between margin width and misclassification — a small C gives a wider margin but tolerates more misclassified points (higher bias), while a large C gives a narrower margin that fits the training data more tightly (lower bias, higher variance). γ (used by RBF, polynomial, and sigmoid kernels) controls how much influence a single training point has — a large γ means a more localized, complex boundary. For the polynomial kernel, **degree** controls the order of the polynomial feature interactions used.

Advantages: Effective in high-dimensional feature spaces (such as the 57-dimensional Spambase feature set), and kernels allow it to model non-linear boundaries.

Limitations: Less interpretable than Logistic Regression, and hyperparameter tuning (kernel, C , γ) is more expensive since it typically requires cross-validated grid or random search.

7 Experimental Setup

Python Version	3.13 (inferred from the notebook's execution environment path)
Scikit-Learn Version	Not explicitly printed in the executed notebook
IDE	Jupyter Notebook
Operating System	Windows (inferred from notebook file paths)
Hardware	Not specified in the notebook

8 Hyperparameter Selection

8.1 Logistic Regression

Search space: $\text{penalty} \in \{\text{L1}, \text{L2}\}$, $C \in \{0.01, 0.1, 1, 10, 100\}$, $\text{solver} \in \{\text{liblinear}, \text{saga}\}$, searched with `GridSearchCV` (5-fold cross-validation, `scoring="accuracy"`).

Parameter	Value
Search Method	<code>GridSearchCV</code> (<code>cv=5</code> , <code>scoring=accuracy</code>)
Best Parameters	$C = 10$, $\text{penalty} = \text{L1}$, $\text{solver} = \text{saga}$
Best CV Accuracy	0.9239 (92.39%)

The selection of L1 regularization with a relatively large $C = 10$ (weak regularization) indicates that most of the 57 features carry useful signal and only weak shrinkage was needed to generalize well; several `ConvergenceWarning` messages were raised for the `saga` solver, indicating that convergence within the `max_iter=1000` limit was borderline for some folds/hyperparameter combinations.

8.2 Support Vector Machine

Search space: kernel-specific grids for Linear ($C \in \{0.1, 1, 10, 100\}$), RBF and Sigmoid ($C \in \{0.1, 1, 10, 100\}$, $\gamma \in \{\text{scale}, \text{auto}\}$), and Polynomial ($C \in \{0.1, 1, 10, 100\}$, $\gamma \in \{\text{scale}, \text{auto}\}$, $\text{degree} \in \{2, 3, 4\}$), searched with `GridSearchCV` (5-fold cross-validation, `scoring="accuracy"`).

Parameter	Value
Search Method	GridSearchCV (cv=5, scoring=accuracy)
Best Parameters	$C = 10$, $\gamma = \text{scale}$, kernel = RBF
Best CV Accuracy	0.9332 (93.32%)

The RBF kernel with a moderate $C = 10$ and the default `scale` setting for γ achieved the best cross-validated accuracy among all SVM configurations searched, slightly ahead of the Logistic Regression search (92.39%).

Hyperparameter Tuning Results Summary

Model	Search Method	Best Parameters	Best CV Accuracy
Logistic Regression	GridSearchCV	$C = 10$, L1, saga	0.9239
SVM	GridSearchCV	$C = 10$, $\gamma = \text{scale}$, RBF	0.9332

9 Results

9.1 Baseline Logistic Regression

The baseline model (`LogisticRegression(max_iter=1000)`, default L2 penalty, $C = 1$) was trained on the scaled features.

Metric	Value
Accuracy	0.9294
Precision	0.9209
Recall	0.8981
F1-score	0.9093
ROC-AUC	Not computed (notebook used hard predictions only)
Training Time (s)	0.0275

Classification Report (Baseline Logistic Regression):

	precision	recall	f1-score	support
0	0.93	0.95	0.94	558
1	0.92	0.90	0.91	363
accuracy			0.93	921
macro avg	0.93	0.92	0.93	921
weighted avg	0.93	0.93	0.93	921

9.2 Tuned Logistic Regression Results

Using the best parameters from GridSearchCV ($C = 10$, L1, saga):

Metric	Value
Accuracy	0.9305
Precision	0.9211
Recall	0.9008
F1-score	0.9109
ROC-AUC	Not computed (notebook used hard predictions only)
Training Time (s)	2.1080

Classification Report (Tuned Logistic Regression):

	precision	recall	f1-score	support
0	0.94	0.95	0.94	558
1	0.92	0.90	0.91	363
accuracy			0.93	921
macro avg	0.93	0.93	0.93	921
weighted avg	0.93	0.93	0.93	921

Tuning improved accuracy only marginally ($0.9294 \rightarrow 0.9305$) and F1-score ($0.9093 \rightarrow 0.9109$), but training time increased substantially ($0.0275\text{s} \rightarrow 2.1080\text{s}$), due to the `saga` solver combined with the L1 penalty requiring more iterations to converge (see the `ConvergenceWarnings` in Section 9.1).

9.3 SVM Kernel-wise Performance

All four kernels were trained with default hyperparameters (`degree=3` for polynomial) on the same scaled train/test split.

Kernel	Accuracy	F1 Score	Training Time (s)
Linear	0.9294	0.9093	0.2931
Polynomial	0.7796	0.6220	0.2591
RBF	0.9273	0.9055	0.1555
Sigmoid	0.8849	0.8528	0.1638

The Linear and RBF kernels performed comparably well (accuracy ≈ 0.93), suggesting the Spambase classes are close to linearly separable in the scaled 57-dimensional feature space. The default Polynomial kernel (degree 3) performed markedly worse (accuracy 0.7796, F1 0.6220), indicating the default polynomial mapping is a poor fit without further tuning. The Sigmoid kernel was intermediate (accuracy 0.8849).

9.4 Best SVM Evaluation

Using the best parameters from `GridSearchCV` ($C = 10$, $\gamma = \text{scale}$, `kernel=RBF`):

Metric	Value
Accuracy	0.9207
Precision	0.9143
Recall	0.8815
F1-score	0.8976
ROC-AUC	Not computed (notebook used hard predictions only)
Training Time (s)	0.1599

Classification Report (Best SVM):

	precision	recall	f1-score	support
0	0.92	0.95	0.94	558
1	0.91	0.88	0.90	363
accuracy			0.92	921
macro avg	0.92	0.91	0.92	921
weighted avg	0.92	0.92	0.92	921

The confusion matrix for this model is shown in Figure 5 (Section 16).

9.5 Model Comparison

Comparison of the two final tuned models on the held-out test set:

Model	Accuracy	Precision	Recall	F1 Score	Training Time (s)
Tuned Logistic Regression	0.9305	0.9211	0.9008	0.9109	2.1080
Best Tuned SVM	0.9207	0.9143	0.8815	0.8976	0.1599

On this test split, Tuned Logistic Regression slightly outperformed the Best Tuned SVM on every classification metric, despite the SVM having achieved a higher cross-validated accuracy during grid search (0.9332 vs 0.9239 — see Section 9). Logistic Regression's training time (2.108s) was, in this run, notably *higher* than the SVM's (0.160s), which is attributable to the slow-converging `saga`+L1 combination rather than to Logistic Regression being inherently more expensive.

9.6 5-Fold Cross-Validation Results

5-fold cross-validation (`cross_val_score`, `cv=5`, `scoring="accuracy"`) was performed on the scaled training set for the best Logistic Regression model and the best SVM model.

Fold	Logistic Regression	SVM
Fold 1	0.9402	0.9429
Fold 2	0.9158	0.9321
Fold 3	0.9226	0.9334
Fold 4	0.9171	0.9212
Fold 5	0.9253	0.9361
Average	0.9242	0.9332

Across all 5 folds, the SVM consistently achieved equal or higher accuracy than Logistic Regression, giving it a higher average cross-validation accuracy (0.9332 vs 0.9242). This contrasts with the single held-out test split in Section 15.5, where Logistic Regression scored higher — an indication that the two models are close in performance and that the ranking between them is somewhat sensitive to the particular train/test split.

10 Result Visualization

Include all graphs generated during the experiment.

For **every graph**, provide:

- Figure
- Caption
- 3–5 line inference

The Class Distribution and Correlation Heatmap figures generated during EDA are presented in Section 4 and are not duplicated here. This section presents the confusion matrices generated during model evaluation. The notebook did not compute prediction probabilities/decision scores, so ROC curves, Precision-Recall curves, a Learning Curve, and ROC-AUC values were not generated as part of this experiment and are therefore not reported here (see Section 19 for discussion).

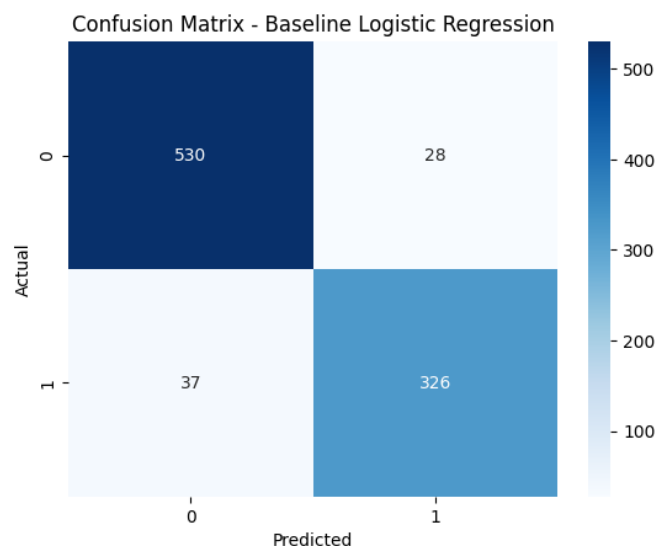


Figure 3: Confusion Matrix — Baseline Logistic Regression.

Inference:

The baseline model correctly classifies the large majority of both classes. Ham emails (class 0) are classified slightly more reliably than spam emails (class 1), consistent with the recall of 0.8981 for the spam class reported in the classification report — a small number of spam emails are still misclassified as ham (false negatives), which is the more costly error type for a spam filter.

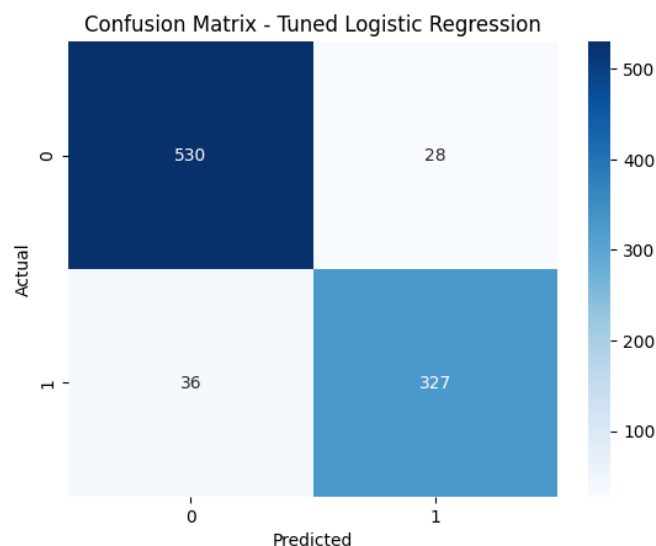


Figure 4: Confusion Matrix — Tuned Logistic Regression.

Inference:

After hyperparameter tuning, the confusion matrix is very similar to the baseline, with a marginally improved spam recall (0.9008 vs 0.8981). This matches the small overall accuracy gain observed ($0.9294 \rightarrow 0.9305$), confirming that tuning refined the decision boundary only slightly rather than substantially changing the model's behaviour.

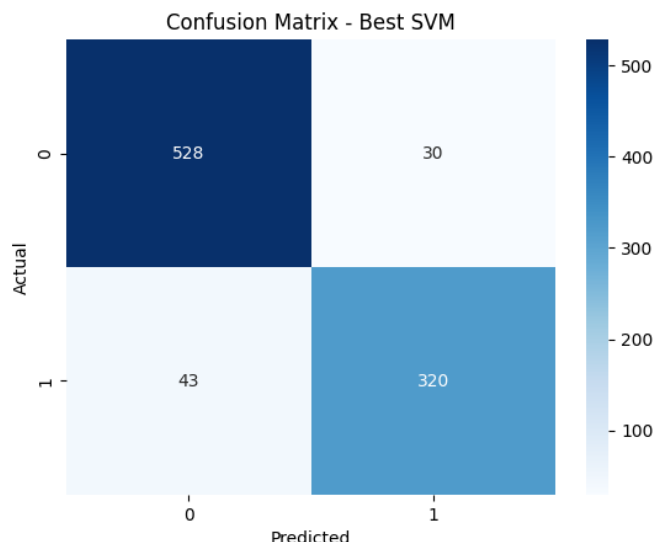


Figure 5: Confusion Matrix — Best SVM (RBF kernel, $C = 10$).

Inference:

The best SVM model shows a slightly higher number of false negatives (spam misclassified as ham) compared to Tuned Logistic Regression, consistent with its lower recall for the spam class (0.8815). Ham emails continue to be classified more reliably than spam emails across all three models, suggesting the ham class is, on average, more distinctly separated in feature space.

11 Discussion

11.1 Overfitting and Underfitting Analysis

For Logistic Regression, the gap between the 5-fold cross-validation accuracy (0.9242 average) and the held-out test accuracy (0.9305) is small (<1 percentage point), which does not indicate overfitting. The selection of a relatively large $C = 10$ (weak regularization) by GridSearchCV, rather than a small C , further supports that the model is not overly constrained and is not showing strong bias either.

For SVM, the default Polynomial kernel (degree 3) clearly **underfits** the data — its test accuracy (0.7796) and F1-score (0.6220) are far below the Linear and RBF kernels, showing that an untuned cubic polynomial mapping is not well matched to this feature space. After tuning, the selected RBF kernel achieved the highest cross-validation accuracy among all SVM configurations (0.9332), and its test performance (0.9207) is reasonably close to this, again suggesting no severe overfitting for the tuned SVM.

11.2 Bias-Variance / Comparative Analysis

Best-performing classifier: On the single held-out test split used in this experiment, **Tuned Logistic Regression** achieved the highest test accuracy (0.9305), precision (0.9211), recall (0.9008), and F1-score (0.9109) among the two final tuned models. However, the **Best SVM** achieved a higher average 5-fold cross-validation accuracy (0.9332 vs 0.9242), so the two models are considered comparably strong, with the ranking depending on which evaluation protocol is used.

Impact of regularization: GridSearchCV selected L1 regularization with a large $C = 10$ for Logistic Regression, i.e. weak regularization. This indicates that most of the 57 Spambase features contribute useful, non-redundant signal, and that the L1 penalty’s implicit feature selection was more effective here than L2’s uniform shrinkage — though the accuracy difference between the tuned and baseline (default L2, $C = 1$) models is small (0.9294 vs 0.9305), meaning regularization choice had only a modest effect for this dataset.

Kernel behaviour in SVM: Linear and RBF kernels performed almost identically well (accuracy 0.9294 and 0.9273 respectively, with default hyperparameters), which is strong evidence that the Spambase classes are close to linearly separable after scaling — the RBF kernel’s extra non-linear flexibility provided little additional benefit over the Linear kernel. The Polynomial kernel performed poorly by default, and the Sigmoid kernel was intermediate, showing that not all kernels are well suited to this dataset without careful tuning.

Bias-variance trade-off: Logistic Regression is a high-bias, low-variance linear model; the fact that it matches or exceeds the more flexible SVM (RBF/kernel) models on this dataset indicates the true decision boundary between spam and ham is close to linear, so the additional model complexity offered by non-linear kernels does not translate into a meaningfully better model here, and would be more prone to variance/overfitting on smaller or noisier datasets.

11.3 General Discussion

Strengths of Logistic Regression: Fast baseline training (0.0275s), interpretable coefficients, and the highest test-set accuracy/F1 among the tuned models in this run.

Limitation observed: the tuned model’s training time increased substantially (2.108s) due to slow `saga` convergence with the L1 penalty (multiple `ConvergenceWarnings` were raised), even though the accuracy gain over the baseline was small.

Strengths of SVM: Fast training across all kernels (0.16s–0.29s), the highest average cross-validation accuracy (0.9332) among all models tuned, and kernel flexibility. **Limitation observed:** performance is highly kernel-dependent — the default Polynomial kernel underperformed substantially (accuracy 0.7796), showing that SVM results are sensitive to kernel choice and hyperparameter tuning in a way that Logistic Regression is not.

Effect of hyperparameter tuning: Tuning improved Logistic Regression’s test accuracy by about 0.1 percentage points and its CV-estimated accuracy could not be directly compared to the baseline (only the tuned model’s CV score was computed). For SVM, tuning (selecting RBF, $C = 10$) raised the cross-validation accuracy to 0.9332 from the untuned RBF-kernel test accuracy of 0.9273, and, more importantly, avoided the substantial performance drop associated with the untuned Polynomial kernel.

Practical interpretation: Both models are viable, accurate spam classifiers on this dataset (all tuned/well-performing configurations scored 90–93% accuracy). Given its interpretability, competitive accuracy, and (aside from the slow `saga` solver run) generally low computational cost, Logistic Regression is a reasonable default choice for this task, with RBF-kernel SVM as a close, slightly more expensive-to-train alternative.

12 Conclusion

This experiment implemented and compared two binary classifiers — Logistic Regression and Support Vector Machine — for spam/ham classification on the 4601-sample Spam-base dataset. A baseline Logistic Regression model achieved 92.94% test accuracy, and after GridSearchCV tuning (L1 penalty, $C = 10$, saga solver) this improved marginally to 93.05% accuracy (91.09% F1-score). Four SVM kernels were compared with default hyperparameters: Linear (92.94%) and RBF (92.73%) performed strongly, Sigmoid was intermediate (88.49%), and Polynomial performed poorly (77.96%). After GridSearchCV tuning, the best SVM (RBF kernel, $C = 10$, $\gamma = \text{scale}$) achieved 93.32% average 5-fold cross-validation accuracy and 92.07% test accuracy. On the held-out test set, Tuned Logistic Regression obtained the best overall metrics (Accuracy 0.9305, F1 0.9109), while Best SVM obtained the best average cross-validation accuracy (0.9332). Overall, the close performance of the linear (Logistic Regression, Linear-kernel SVM) and non-linear (RBF SVM) models indicates the Spambase classes are close to linearly separable, and hyperparameter tuning provided a modest but genuine improvement over the baseline/default configurations for both model families.

13 References

- [1] scikit-learn developers, “Logistic Regression,” scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
- [2] scikit-learn developers, “Support Vector Machines,” scikit-learn documentation. [Online]. Available: <https://scikit-learn.org/stable/modules/svm.html>
- [3] scikit-learn developers, “Tuning the hyper-parameters of an estimator,” scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/grid_search.html
- [4] Kaggle, “Spambase Dataset.” [Online]. Available: <https://www.kaggle.com/datasets/somesh24/spambase>
- [5] UCI Machine Learning Repository, “Spambase Data Set.” [Online]. Available: <https://archive.ics.uci.edu/dataset/94/spambase>